Mathematical Reasoning in Small Open-Weight LLMs: Prompting Accuracy and Response Efficiency

Baizheng Chen
Department of Computer Science
Student ID: 72510800

***Abstract*—This report studies Topic 1 of the CS6493 group project: mathematical reasoning ability of large language models. We evaluate two small open-weight mathematical reasoning models, Qwen2.5-Math-1.5B and DeepSeek-R1-Distill-Qwen-1.5B, on MATH-500, GSM8K, and AIME 2024. The method set includes four assignment-aligned baselines, Chain-of-Thought zero-shot, Chain-of-Thought few-shot, Self-Refine, and Self-Consistency, plus two extensions: Plan-and-Solve as the main prompt-only method and Tool-Integrated Reasoning as an optional tool-augmented method. TIR is also interpreted as a lightweight agentic reasoning pattern because the model can decide when to call a tool, observe the result, and continue solving. In addition to accuracy and final response length, we report total generated length and a behavior-aware joint score that gates on correctness and penalizes excessive generation, repeated answer signals, and excessive reflection.**

I. Introduction

Large language models can produce fluent step-by-step solutions, yet mathematical reasoning remains difficult because the model must parse symbols, execute multi-step logic, and return a precise final answer. The project requirement asks us to compare prompt-based methods on MATH-500, GSM8K, and AIME 2024 using two small target models, with accuracy and response length as key metrics.

Our study follows this requirement but adds an efficiency perspective. A method that reaches the right answer after very long or repetitive reasoning is not equivalent to a method that reaches the same answer concisely. This distinction is especially important for Self-Consistency, Self-Refine, Plan-and-Solve, and tool-augmented methods, where one problem can trigger multiple generations or tool rounds.

The main extension is `plan_solve`. It is a pure prompt-only method, so it is directly comparable with Chain-of-Thought (CoT), Self-Refine, and Self-Consistency. We also include `tir`, Tool-Integrated Reasoning (TIR), as a separate tool-augmented extension. TIR is reported with an explicit caveat because it changes the interaction setting by allowing Python execution, but this change is also its main research value: it moves mathematical reasoning toward an agentic loop of reasoning, tool invocation, observation, and continuation.

II. Related Work

CoT prompting elicits intermediate reasoning steps before the final answer [1]. Self-Consistency improves robustness by sampling multiple reasoning paths and aggregating answers [2], while Self-Refine asks the model to critique and revise its own output [3]. Plan-and-Solve prompting first asks the model to decompose the task and then solve according to the plan, targeting missing-step errors in zero-shot reasoning [4].

Tool-integrated approaches combine natural-language reasoning with executable computation. ToRA shows that tool-use trajectories can substantially improve mathematical problem solving by interleaving reasoning with external computational tools [5]. Our TIR implementation is not a trained ToRA model; it is a prompt-level tool loop that asks the model to emit Python snippets when useful. This makes TIR a small agentic workflow rather than only a longer prompt: the model alternates between deciding whether a tool is needed, producing an executable action, receiving an observation, and revising the remaining solution.

The datasets and models are also grounded in prior work. GSM8K tests grade-school mathematical word problems [6], while MATH provides competition-style problems [7]. Qwen2.5-Math is a math-specialized Qwen model family [8]; DeepSeek-R1-Distill-Qwen models are dense distilled reasoning models released with DeepSeek-R1 [9]. Finally, recent reasoning-efficiency work motivates our reflection and answer-behavior metrics: NoWait studies the cost of explicit reflection tokens such as "Wait" [10], and Dynamic Early Exit studies answer-aware truncation for long reasoning sequences [11]. We use these ideas only as metric-design motivation, not as a claim that our system implements decoding-time early exit.

III. Methodology

A. Models and Data

We evaluate Qwen2.5-Math-1.5B and DeepSeek-R1-Distill-Qwen-1.5B. The datasets are fixed sampled subsets stored in the project: 50 samples each for MATH-500 and GSM8K, and 30 samples for AIME 2024. The total per model-method run is therefore 130 questions. This fixed-sample design improves reproducibility, but it also introduces sampling bias. As a result, the reported metrics should be interpreted as reference values for relative comparison within this project, not as unbiased estimates of full-dataset performance.

B. Prompting Methods

Table I defines the six reported methods and separates the dialogue pattern from the prompt idea.

cot_zero and cot_few_shot are single-turn methods. self_consistency is not an iterative dialogue, but it is multi-sample because it launches five independent single-turn generations. self_refine, plan_solve, and tir are multi-pass methods; TIR is also an agentic tool loop because Python execution can be inserted between model calls and the tool output becomes an observation for the next reasoning step.

These six methods were chosen to cover three different intervention styles. First, cot_zero and cot_few_shot modify only the prompt context and therefore provide the cleanest baseline for comparison. Second, self_refine, self_consistency, and plan_solve change the internal structure of inference by adding critique, resampling, or decomposition. Third, tir changes the boundary of the system itself by allowing the model to call an external executor. This categorization matters because two methods can have similar accuracy improvements while relying on very different additional computation budgets.

The choice of plan_solve as the main proposed prompt-only method is intentional. Compared with self_refine, it adds structure before the solution rather than after it, which is a better match for mathematical tasks where missing a step early can derail the entire derivation. Compared with self_consistency, it does not rely on repeated sampling and voting, so it preserves a cleaner comparison under limited compute. In other words, Plan-and-Solve is attractive not only because it improves accuracy, but because its mechanism is easy to explain and its extra cost is conceptually transparent.

The role of tir is different. It is included not as a replacement for prompt-only comparison, but as a contrast case that shows what happens when a small reasoning model is allowed to externalize arithmetic and symbolic operations. This distinction is useful for the project because mathematical reasoning often mixes two sources of failure: reasoning-plan errors and execution errors. Prompt decomposition can address the first kind, while tool use can directly reduce the second. Reporting both methods therefore clarifies which gains come from better prompting and which come from changing the overall problem-solving loop.

C. Metrics

Accuracy is the primary metric. A sample is correct when the parsed prediction is mathematically equal to the normalized ground truth. Response length is separated into final response length and total generated length. Final response length is the text submitted to the grader. Total generated length includes intermediate generations: plan plus solve for Plan-and-Solve, solve plus critique plus refine for Self-Refine, all samples for Self-Consistency, and the full tool transcript for TIR. When complete token-count metadata is available, total generated tokens use that recorded generation length, including generated code and recorded reasoning text; otherwise token length is approximated as $\lceil \max(\text{characters}/4, \text{words}/0.75) \rceil$. Character length is the raw string length.

TABLE I
PROMPTING METHOD DEFINITIONS.

Method	Dialogue pattern	Definition
cot_zero	Single-turn	Zero-shot Chain-of-Thought. The model receives the problem and is asked to solve step by step, ending with a boxed final answer.
cot_few_shot	Single-turn	Few-shot Chain-of-Thought. The prompt adds three worked examples before the target problem to guide format and reasoning style.
self_refine	Multi-turn	Solve-critique-refine. The model first writes a solution, then critiques it, then rewrites a final improved solution.
self_consistency	Multi-sample	Sample-and-vote CoT. The model samples five independent single-turn solutions; final answers are extracted, majority voted, and the first response matching the voted answer is graded.
plan_solve	Two-turn	Plan-and-Solve. The model first writes a short numbered plan, then solves the problem by following that plan.
tir	Agentic tool-loop	Tool-Integrated Reasoning. The model may emit fenced Python code, observe the returned output block, and continue solving with that evidence.

The distinction between final response length and total generated length is central to this report. A method may look concise if only its final answer is counted, while actually consuming much more computation through planning, refinement, sampling, or tool interaction. For this reason, final response length is treated as a presentation metric, while total generated length is treated as a computation-cost metric. Reporting both avoids giving an unfair advantage to multi-stage methods whose intermediate reasoning is otherwise hidden.

For each sample, correctness gates the final score. The efficiency term is

$$\text{efficiency}_i = \text{answer}_i^{0.5} \text{length}_i^{0.3} \text{reflection}_i^{0.2}. \tag{1}$$

The final sample score is accuracy-first:

$$\text{score}_i = c_i \cdot (0.6 + 0.4 \cdot \text{efficiency}_i), \tag{2}$$

where $c_i \in \{0, 1\}$ is the correctness indicator. Thus incorrect answers still receive zero, while correct answers retain most of their accuracy credit and use efficiency as a secondary modifier. The three factors are defined as follows.

Length factor. Let L_i be total generated tokens. With $L_{\text{ref}} = 1024$ and $\tau_L = 512$,

$$\text{length}_i = \exp\left(-\frac{\max(0, L_i - L_{\text{ref}})}{\tau_L}\right). \tag{3}$$

This penalizes only overlong generations.

Answer factor. We count answer signals such as \boxed{, "final answer is", "the answer is", and "answer:". Let A_i be this count. Let R_i be the tail ratio, defined as the character ratio from the first answer signal to the end of the final response; if no answer signal exists, $R_i = 0$. The answer factor is

$$\text{count}_i = \exp\left(-\max(0, A_i - 1)\right), \tag{4}$$

$$\text{tail}_i = \begin{cases} 1, & A_i < 2, \\ \exp\left(-\frac{\max(0, R_i - 0.2)}{0.2}\right), & A_i \geq 2, \end{cases} \tag{5}$$

$$\text{answer}_i = \text{count}_i \cdot \text{tail}_i. \tag{6}$$

It penalizes repeated answer declarations and long continuation after an early answer.

Reflection factor. We count reflective cues such as "wait", "however", "check", "verify", "rethink", and their Chinese equivalents. Let F_i be this reflection count. With reference count 1 and $\tau_F = 2$,

$$\text{reflection}_i = \exp\left(-\frac{\max(0, F_i - 1)}{2}\right). \tag{7}$$

The reported joint score is the mean of score_i. Mean efficiency on correct samples is reported separately as a diagnostic value.

This scoring design deliberately keeps accuracy dominant. The floor term of 0.6 means that once a sample is correct, most of its score is already secured, and the efficiency term only adjusts the remaining 0.4. This avoids the undesirable behavior of ranking a short wrong answer above a long correct one. At the same time, the secondary factors remain useful because mathematical reasoning outputs often differ dramatically in verbosity and self-correction style even when they achieve the same correctness.

The three factors also target different failure modes. The length factor captures over-generation, especially for multi-sample or iterative methods. The answer factor captures degenerate answer repetition such as repeatedly restating the final answer or continuing to explain after the answer has effectively been given. The reflection factor captures excessive hesitation or recursive self-checking, which is often a symptom of unstable reasoning rather than genuine improvement. None of these factors is intended to replace correctness; instead, they provide a structured way to compare methods that would otherwise look identical under accuracy alone.

This metric design also reflects a practical reporting problem. If only accuracy is shown, `self_consistency` can appear disproportionately attractive because its voting mechanism often raises correctness at the cost of very large hidden generation. If only final answer length is shown, then `plan_solve`, `self_refine`, and `tir` can appear cheaper than they really are, because most of their cost lies in intermediate reasoning. If only total length is shown, then methods that are correct but slightly verbose can be treated too harshly. The current design is therefore a compromise: correctness decides whether a sample deserves credit at all, and efficiency is used only to separate correct samples that solve the task with different levels of overhead.

Another reason to use factorized metrics is interpretability during error analysis. A single scalar can rank methods, but it cannot explain why two methods with similar accuracy receive different joint scores. Factor decomposition does. Low length factor points to over-generation, low answer factor points to unstable answer formatting or repeated conclusion cues, and low reflection factor points to excessive self-correction language. This is especially useful when comparing methods that appear superficially similar, such as `cot_few_shot` and `plan_solve`, because it reveals whether the gain comes from better task decomposition or merely from changing answer style.

D. Experimental Protocol

All experiments follow the same high-level pipeline: run inference on the fixed sampled subsets, extract final answers, compute sample-level metrics, and then aggregate results by model, method, and dataset. The same answer parser and mathematical equivalence grader are used across all methods to reduce evaluation drift caused by formatting differences.

There are still meaningful protocol differences across methods. `cot_zero` and `cot_few_shot` are single-pass and therefore their final length and total length are identical. `self_refine` expands the trajectory into solve, critique, and rewrite stages. `self_consistency` launches five independent solutions and uses majority vote over extracted answers, which substantially increases generation cost even when the final displayed answer is short. `plan_solve` explicitly separates decomposition from execution, and `tir` introduces executable actions and observations into the trajectory. Because of these differences, it is necessary to compare methods not only by end accuracy but also by how much additional generation each design requires.

When more than one evaluated row exists for the same model, method, and dataset, this report keeps the row with the highest joint score. This choice is pragmatic: the final report is intended to summarize the strongest observed configuration for each method under the current project pipeline. However, this also means the tables should be read as best-run summaries rather than as estimates of average run-to-run variance.

IV. Experiments

When multiple evaluated rows are available for the same model, method, and split, the tables report the row with the highest joint score. Overall rows are selected independently from the overall summaries, rather than recomputed from the per-dataset rows.

Table II shows that `plan_solve` is the strongest prompt-only extension in accuracy for both models and has higher joint score than all four baselines. For Qwen2.5-Math-1.5B, it improves overall accuracy from 0.531–0.562 for the prompt-only baselines to 0.785. For DeepSeek-R1-Distill-Qwen-1.5B, the updated `plan_solve` run reaches 0.892 overall accuracy, with strong performance on all three splits. After counting full generated tokens, its total generation cost is no longer as small as final response length alone suggests.

TIR remains highly competitive, reaching the highest overall accuracy for DeepSeek-R1-Distill-Qwen-1.5B at 0.908 and a joint score of 0.7140. For Qwen2.5-Math-1.5B, it reaches 0.800 accuracy and 0.6681 joint score. However, it is not directly comparable as a prompt-only method because it can use Python execution. The correct interpretation is that TIR is a tool-augmented and agentic extension showing how arithmetic and symbolic computation can improve accuracy when the model is allowed to externalize part of the solution process into executable actions, observe results, and continue reasoning. This also incurs additional generation cost from code, tool rounds, and recorded reasoning.

TABLE II
OVERALL RESULTS ACROSS 130 QUESTIONS.

Model	Method	Accuracy	Final Tok.	Total Tok.	Joint
DeepSeek-R1-Distill-Qwen-1.5B	cot_zero	0.785	4100.1	4100.1	0.5112
DeepSeek-R1-Distill-Qwen-1.5B	cot_few_shot	0.785	3534.8	3534.8	0.4996
DeepSeek-R1-Distill-Qwen-1.5B	self_refine	0.792	362.3	4709.8	0.5766
DeepSeek-R1-Distill-Qwen-1.5B	self_consistency	0.854	4217.5	21449.8	0.5214
DeepSeek-R1-Distill-Qwen-1.5B	plan_solve	0.892	174.1	2252.7	0.6955
DeepSeek-R1-Distill-Qwen-1.5B	tir	0.908	167.5	1460.3	0.7140
Qwen2.5-Math-1.5B	cot_zero	0.531	2248.2	2248.2	0.5176
Qwen2.5-Math-1.5B	cot_few_shot	0.515	2533.6	2533.6	0.4604
Qwen2.5-Math-1.5B	self_refine	0.508	348.5	1001.1	0.4726
Qwen2.5-Math-1.5B	self_consistency	0.562	848.9	4524.0	0.4789
Qwen2.5-Math-1.5B	plan_solve	0.785	380.9	721.7	0.6570
Qwen2.5-Math-1.5B	tir	0.800	371.9	537.1	0.6681

TABLE III
ACCURACY BY DATASET.

Model	Method	MATH-500	GSM8K	AIME 2024
DeepSeek-R1-Distill-Qwen-1.5B	cot_zero	0.900	0.900	0.400
DeepSeek-R1-Distill-Qwen-1.5B	cot_few_shot	0.920	0.920	0.333
DeepSeek-R1-Distill-Qwen-1.5B	self_refine	0.880	0.920	0.433
DeepSeek-R1-Distill-Qwen-1.5B	self_consistency	0.920	0.980	0.533
DeepSeek-R1-Distill-Qwen-1.5B	plan_solve	0.940	0.980	0.667
DeepSeek-R1-Distill-Qwen-1.5B	tir	0.960	0.940	0.767
Qwen2.5-Math-1.5B	cot_zero	0.560	0.760	0.100
Qwen2.5-Math-1.5B	cot_few_shot	0.560	0.780	0.000
Qwen2.5-Math-1.5B	self_refine	0.560	0.740	0.033
Qwen2.5-Math-1.5B	self_consistency	0.580	0.840	0.067
Qwen2.5-Math-1.5B	plan_solve	0.860	0.960	0.367
Qwen2.5-Math-1.5B	tir	0.900	0.940	0.400

Table IV explains why efficiency diagnostics are still useful under the accuracy-first score. Self-Consistency can have high accuracy, but its total generated length is much larger because it samples five independent outputs. TIR and Plan-and-Solve retain much shorter final answers, but their total token cost is higher once full generation length is counted. This supports the decision to report both final response length and total generation cost.

The results also show a clear model-specific pattern. DeepSeek-R1-Distill-Qwen-1.5B is consistently stronger than Qwen2.5-Math-1.5B across all six methods, but the gap is not uniform. Under the two proposed methods, both models improve substantially relative to their prompt-only baselines, which suggests that decomposition and tool use are not only helping one particular backbone. At the same time, the absolute gains are larger for DeepSeek-R1-Distill-Qwen-1.5B, indicating that a stronger reasoning prior can better exploit structured prompting and tool interaction.

The dataset breakdown is equally informative. GSM8K is the easiest split for both models and most methods, while AIME 2024 remains the hardest. This is consistent with the intuition that elementary multi-step arithmetic benefits more directly from decomposition and controlled execution than competition-level problems requiring deeper symbolic insight. The AIME numbers therefore act as a stress test: methods that appear strong on GSM8K may still fail to transfer cleanly to harder mathematical reasoning.

Among prompt-only approaches, plan_solve is the most balanced. It improves accuracy while keeping total generation noticeably below self_consistency and often below self_refine. This is an important practical result. A method that adds structure without multiplying trajectory

TABLE IV
OVERALL BEHAVIOR FACTORS ON CORRECT SAMPLES.

Model	Method	Len.	Ans.	Refl.
DS	cot_zero	0.629	0.248	0.017
DS	cot_few	0.727	0.083	0.039
DS	self_refine	0.286	0.648	0.578
DS	self_cons.	0.010	0.160	0.013
DS	plan_solve	0.608	0.368	0.983
DS	tir	0.726	0.339	0.975
Qwen	cot_zero	1.000	0.918	0.959
Qwen	cot_few	1.000	0.585	0.961
Qwen	self_refine	0.912	0.804	0.914
Qwen	self_cons.	0.515	0.758	0.942
Qwen	plan_solve	0.963	0.368	0.979
Qwen	tir	0.972	0.368	0.955

count is easier to justify in a course project because it preserves the prompt-based comparison setting while still producing a visible empirical gain.

The baseline methods expose complementary weaknesses. `cot_zero` and `cot_few_shot` often generate very long answers, especially for DeepSeek-R1-Distill-Qwen-1.5B, which hurts efficiency even when correctness is acceptable. `self_refine` can shorten the final answer but still accumulates substantial hidden generation cost through its critique stage. `self_consistency` improves accuracy most clearly among the baselines, but its token budget is by far the largest, making it a strong example of why accuracy alone is insufficient for evaluating reasoning strategies.

V. DISCUSSION

The safest final extension is Plan-and-Solve. It is easy to justify pedagogically: mathematical problem solving often begins by identifying the relevant steps before executing calculations. It is also fair against other prompt-only methods because it does not use an external calculator or verifier. The updated results show strong accuracy gains for both models, although its total generation cost should be reported separately from its concise final answers.

TIR is more distinctive but requires careful framing. It should not be described as just another prompt-only baseline. It changes the setting by giving the model a Python interpreter, so its gains reflect both reasoning and tool use. This setting is worth emphasizing rather than minimizing: it is a compact agentic pattern where the model chooses an action, receives an observation, and then updates the final solution. For a mathematical reasoning project, this is especially valuable because many errors in small models are arithmetic or symbolic execution errors, exactly the cases where a tool loop can help. It also points to a broader direction beyond static prompting: small mathematical models can be paired with reliable tools to form more capable solver agents.

The behavior-aware metrics are useful because raw accuracy can hide inefficient reasoning. Self-Consistency improves accuracy through sampling, but its total generation cost is high. DeepSeek CoT runs also show extremely long final responses, which hurts efficiency even when the answer is correct. The answer and reflection factors further identify repeated final-answer cues and excessive self-correction language. These metrics should be read as diagnostic signals rather than replacements for accuracy.

There is also a dataset-level caveat. Because all experiments use fixed sampled subsets rather than the full benchmarks, some observed gains or drops may depend on sample composition. The tables are still useful for method comparison under a controlled setting, but the exact numbers should be treated as indicative rather than definitive.

There is a second caveat on metric design. Our score intentionally compresses several behaviors into a single efficiency modifier, which makes reporting concise but also hides some nuance. For example, a method may receive a similar efficiency value because it is slightly too long, because it repeats answer cues, or because it contains many reflection tokens. The factor table is therefore necessary context whenever joint score is discussed. In other words, the scalar score is useful for ranking, but the factor decomposition is needed for interpretation.

A final methodological takeaway is that prompt design and evaluation design should be discussed together. Once methods become multi-stage, final answer length is no longer a faithful proxy for actual reasoning cost. Likewise, once a method can call tools, raw accuracy becomes harder to interpret unless the interaction setting is stated explicitly. The report therefore treats method definition, computation accounting, and score design as one connected experimental choice rather than three unrelated implementation details.

VI. CONCLUSION

We evaluated two small open-weight mathematical reasoning models on MATH-500, GSM8K, and AIME 2024. The main recommendation is to present Plan-and-Solve as the primary proposed prompt-only method because it is fair, simple, and well aligned with mathematical problem solving. TIR should be presented as an optional tool-augmented and agentic extension with strong empirical results and an explicit caveat about Python execution. The final report should emphasize both accuracy and response efficiency, separating final answer length from total generated cost.

REFERENCES

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022.

[2] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.

[3] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, 2024.

[4] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," *arXiv preprint arXiv:2305.04091*, 2023.

[5] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, "ToRA: A tool-integrated reasoning agent for mathematical problem solving," *arXiv preprint arXiv:2309.17452*, 2023.

[6] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[7] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring mathematical problem solving with the MATH dataset," in *Advances in Neural Information Processing Systems*, 2021.

[8] A. Yang, B. Zhang, B. Hui, B. Gao, B. Yu, C. Li, D. Liu, J. Tu, J. Zhou, J. Lin, K. Lu, M. Xue, R. Lin, T. Liu, X. Ren, Z. Zhang *et al.*, "Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement," *arXiv preprint arXiv:2409.12122*, 2024.

[9] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[10] C. Wang, Y. Feng, D. Chen, Z. Chu, R. Krishna, and T. Zhou, "Wait, we don't need to "wait"! removing thinking tokens improves reasoning efficiency," *arXiv preprint arXiv:2506.08343*, 2025.

[11] C. Yang, Q. Si, Y. Duan, Z. Zhu, C. Zhu, Q. Li, M. Chen, Z. Lin, and W. Wang, "Dynamic early exit in reasoning models," *arXiv preprint arXiv:2504.15895*, 2025.